

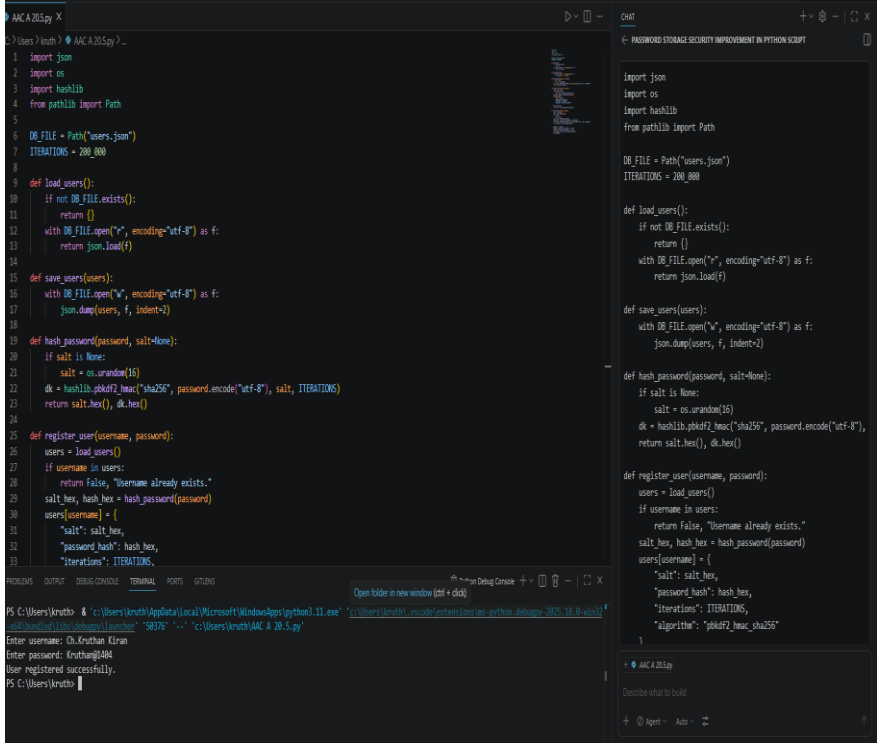
Name: A Harika

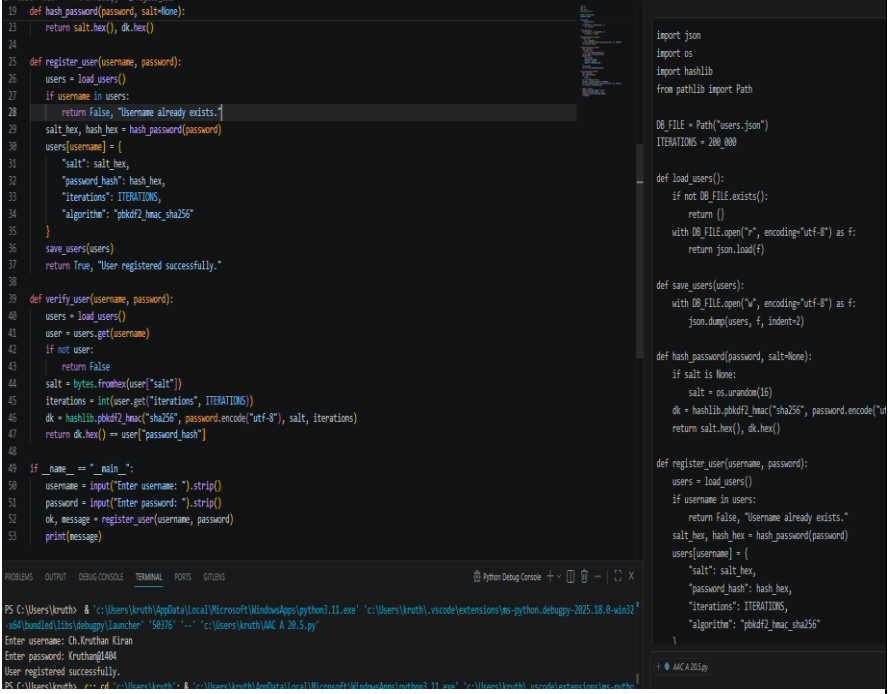
H.NO:2303A51113

BATCH:26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M.Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
		Mr. Bingi Raju	
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week10 – Friday	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number:20.5(Present assignment number)/24(Total number of assignments)			

Q.No.	Question	Expected Time to complete
1	Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code	Week10 - Friday

	Lab Objectives: - Understand how to test AI-generated code for common security vulnerabilities. - Learn to apply secure coding principles while analyzing AI outputs. - Practice detecting risks such as SQL injection, XSS, hardcoded credentials, and weak encryption. - Enhance code reliability and safety by using AI for secure refactoring.	
	Task 1 – Password Storage Vulnerability Check Task: Analyze an AI-generated signup script for insecure password storage. Instructions: - Prompt AI to generate a Python user registration program. - Check whether passwords are stored in plain text. - Improve the script using hashing (bcrypt or hashlib). Expected Output: - A secure registration script with hashed password storage. 	

	 <pre>19 def hash_password(password, salt=None): 20 return salt.hex(), dk.hex() 21 22 23 24 25 def register_user(username, password): 26 users = load_users() 27 if username in users: 28 return False, "Username already exists." 29 salt_hex, hash_hex = hash_password(password) 30 users[username] = { 31 "salt": salt_hex, 32 "password_hash": hash_hex, 33 "iterations": ITERATIONS, 34 "algorithm": "pbkdf2_hmac_sha256" 35 } 36 save_users(users) 37 return True, "User registered successfully." 38 39 40 def verify_user(username, password): 41 users = load_users() 42 user = users.get(username) 43 if not user: 44 return False 45 salt = bytes.fromhex(user["salt"]) 46 iterations = int(user.get("iterations", ITERATIONS)) 47 dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, iterations) 48 return dk.hex() == user["password_hash"] 49 50 if __name__ == "__main__": 51 username = input("Enter username: ").strip() 52 password = input("Enter password: ").strip() 53 ok, message = register_user(username, password) 54 print(message) 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100</pre> <pre>import json import os import hashlib from pathlib import Path DB_FILE = Path("users.json") ITERATIONS = 200_000 def load_users(): if not DB_FILE.exists(): return [] with DB_FILE.open("r", encoding="utf-8") as f: return json.load(f) def save_users(users): with DB_FILE.open("w", encoding="utf-8") as f: json.dump(users, f, indent=2) def hash_password(password, salt=None): if salt is None: salt = os.urandom(16) dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, ITERATIONS) return salt.hex(), dk.hex() def register_user(username, password): users = load_users() if username in users: return False, "Username already exists." salt_hex, hash_hex = hash_password(password) users[username] = { "salt": salt_hex, "password_hash": hash_hex, "iterations": ITERATIONS, "algorithm": "pbkdf2_hmac_sha256" } save_users(users) return True, "User registered successfully."</pre> <pre>PS C:\Users\kruth> & "c:\Users\kruth\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:\Users\kruth\.vscode\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\libs\debugpy\launcher" "58376" "-." "c:\Users\kruth\AAC A 20.5.py" Enter username: Ch.Kruthan Kiran Enter password: Kruthan@1484 User registered successfully. PS C:\Users\kruth></pre>	
	Task 2 – Hardcoded Secrets Detection Task: Evaluate AI-generated code for hardcoded API keys or credentials. Instructions: • Ask AI to generate a script that connects to an external API. • Identify if API keys/passwords are written directly in code. • Refactor using environment variables or config files. Expected Output: • A secure script with secrets managed properly.	

```
AKA205.py X
C:\Users\kruth> AKA205.py get_weather
1 import os
2 import json
3 from pathlib import Path
4 import requests
5
6 BASE_URL = os.getenv("WEATHER_API_BASE_URL", "https://api.openweathermap.org/data/2.5/weather")
7 CONFIG_PATH = Path(__file__).with_name("weather_config.json")
8
9 def load_api_key() -> str | None:
10     env_key = os.getenv("WEATHER_API_KEY")
11     if env_key:
12         return env_key
13     if CONFIG_PATH.exists():
14         with CONFIG_PATH.open("r", encoding="utf-8") as f:
15             data = json.load(f)
16             key = data.get("WEATHER_API_KEY")
17             if key:
18                 return key
19     return None
20
21 def get_weather(city: str) -> dict:
22     api_key = load_api_key()
23     if not api_key:
24         api_key = input("Enter WEATHER_API_KEY: ").strip()
25     if not api_key:
26         raise RuntimeError("Missing WEATHER_API_KEY in environment, weather_config.json, and runtime input")
27     params = {"q": city, "appid": api_key, "units": "metric"}
28     response = requests.get(BASE_URL, params=params, timeout=15)
29     if response.status_code == 401:
30         raise RuntimeError("Invalid WEATHER_API_KEY. Please verify or generate a new key.")
31     response.raise_for_status()
32     return response.json()
33
34 if __name__ == "__main__":
35     try:
36         city = input("City: ").strip()
37         data = get_weather(city)
38         print(data)
39     except requests.exceptions.RequestException as e:
40         print(f"Network/API error: {e}")
41     except RuntimeError as e:
42         print(e)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\kruth> & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '55964' '-' 'c:\Users\kruth\AKA 20.5.py'

raise HTTPError(http_error_msg, response=self)

requests.exceptions.HTTPError: 401 Client Error: Unauthorized for url: https://api.openweathermap.org/data/2.5/weather?q=Parkal&appid=84f61c6269619ae6348e093b0aa4209a&units=metric

PS C:\Users\kruth> cd 'c:\Users\kruth'; & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50806' '-' 'c:\Users\kruth\AKA 20.5.py'

City: Parkal

Enter WEATHER_API_KEY: 84f61c6269619ae6348e093b0aa4209a

Invalid WEATHER_API_KEY. Please verify or generate a new key.

```
AAC A 20.5.py X
C:\Users\kruth> AAC A 20.5.py get_weather
21 def get_weather(city: str) -> dict:
22     api_key = load_api_key()
23     if not api_key:
24         api_key = input("Enter WEATHER_API_KEY: ").strip()
25     if not api_key:
26         raise RuntimeError("Missing WEATHER_API_KEY in environment, weather_config.json, and runtime input")
27     params = {"q": city, "appid": api_key, "units": "metric"}
28     response = requests.get(BASE_URL, params=params, timeout=15)
29     if response.status_code == 401:
30         raise RuntimeError("Invalid WEATHER_API_KEY. Please verify or generate a new key.")
31     response.raise_for_status()
32     return response.json()
33
34 if __name__ == "__main__":
35     try:
36         city = input("City: ").strip()
37         data = get_weather(city)
38         print(data)
39     except requests.exceptions.RequestException as e:
40         print(f"Network/API error: {e}")
41     except RuntimeError as e:
42         print(e)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\kruth> & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '55964' '-' 'c:\Users\kruth\AAC A 20.5.py'

raise HTTPError(http_error_msg, response=self)

requests.exceptions.HTTPError: 401 Client Error: Unauthorized for url: https://api.openweathermap.org/data/2.5/weather?q=Parkal&appid=84f61c6269619ae6348e093b0aa4209a&units=metric

PS C:\Users\kruth> cd 'c:\Users\kruth'; & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50806' '-' 'c:\Users\kruth\AAC A 20.5.py'

City: Parkal

Enter WEATHER_API_KEY: 84f61c6269619ae6348e093b0aa4209a

Invalid WEATHER_API_KEY. Please verify or generate a new key.

Task 3 – File Upload Security Validation

Task:

Test an AI-generated file upload program for unsafe file handling.

Instructions:

- Prompt AI to generate a Flask file upload example.

- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename handling.

Expected Output:

- A secure file upload implementation.

```

AAC A 20.5.py
C:\Users\kruth> AAC A 20.5.py
1 from flask import Flask, request, jsonify
2 from werkzeug.utils import secure_filename
3 from pathlib import Path
4 import uuid
5
6 app = Flask(__name__)
7 app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024
8
9 UPLOAD_DIR = Path("uploads")
10 UPLOAD_DIR.mkdir(exist_ok=True)
11
12 ALLOWED_EXTENSIONS = ("png", "jpg", "jpeg", "pdf", "txt")
13 EXTENSION_TO_KIND = {
14     "png": ("img"),
15     "jpg": ("img"),
16     "jpeg": ("img"),
17     "pdf": ("pdf"),
18     "txt": ("txt"),
19 }
20
21 def detect_kind(data: bytes) -> str:
22     if data.startswith(b"\xff\xd9\xff"):
23         return "img"
24     if data.startswith(b"\xff\xff\xff"):
25         return "img"
26     if data.startswith(b"\x50\x41\x52"):
27         return "pdf"
28     if data.startswith(b"\x25\x50\x44"):
29         return "txt"
30     if data.startswith(b"\x50\x41\x52"):
31         return "zip"
32     if b"b" in data:
33         return "binary"
34
35 PS C:\Users\kruth> cd 'c:\Users\kruth' & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-
ebugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57474' '-.' 'c:\Users\kruth\AAC A 20.5.py'
ModuleNotFoundError: No module named 'flask'
PS C:\Users\kruth> cd 'c:\Users\kruth' & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-
ebugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57510' '-.' 'c:\Users\kruth\AAC A 20.5.py'
* Serving Flask app 'AAC A 20.5'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit

```

```

PASSWORD STORAGE SECURITY IMPROVEMENT IN PYTHON SCRIPT
1 from flask import Flask, request, jsonify
2 from werkzeug.utils import secure_filename
3 from pathlib import Path
4 import uuid
5
6 app = Flask(__name__)
7 app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024
8
9 UPLOAD_DIR = Path("uploads")
10 UPLOAD_DIR.mkdir(exist_ok=True)
11
12 ALLOWED_EXTENSIONS = ("png", "jpg", "jpeg", "pdf", "txt")
13 EXTENSION_TO_KIND = {
14     "png": ("img"),
15     "jpg": ("img"),
16     "jpeg": ("img"),
17     "pdf": ("pdf"),
18     "txt": ("txt"),
19 }
20
21 def detect_kind(data: bytes) -> str:
22     if data.startswith(b"\xff\xd9\xff"):
23         return "img"
24     if data.startswith(b"\xff\xff\xff"):
25         return "img"
26     if data.startswith(b"\x50\x41\x52"):
27         return "pdf"
28     if data.startswith(b"\x25\x50\x44"):
29         return "txt"
30     if data.startswith(b"\x50\x41\x52"):
31         return "zip"
32     if b"b" in data:
33         return "binary"
34     try:
35         data.decode("utf-8")
36     except UnicodeDecodeError:
37         return "unknown"
38
39 @app.post("/upload")
40 def upload_file():
41     if "file" not in request.files:
42         return jsonify({"error": "No file part"}), 400
43
44     file = request.files["file"]
45     if file.filename is None or file.filename.strip() == "":
46         return jsonify({"error": "No selected file"}), 400
47
48     original_name = secure_filename(file.filename)
49     if original_name == "":
50         return jsonify({"error": "Invalid filename"}), 400
51
52     ext = Path(original_name).suffix.lower().lstrip(".")
53     if ext not in ALLOWED_EXTENSIONS:
54         return jsonify({"error": "File extension not allowed"}), 400
55
56     head = file.stream.read(8192)
57     if not head:
58         return jsonify({"error": "Empty file"}), 400
59
60     kind = detect_kind(head)
61     if kind in ("exe", "zip", "binary", "unknown"):
62         return jsonify({"error": "Unsafe or unsupported file content"}), 400
63
64     if kind not in EXTENSION_TO_KIND[ext]:
65

```

```
AAC A 20.5.py 2 X
C:\Users\kruth > AAC A 20.5.py > ...
41 def upload_file():
50     if original_name == "":
51         return jsonify({"error": "Invalid filename"}), 400
52
53     ext = Path(original_name).suffix.lower().lstrip(".")
54     if ext not in ALLOWED_EXTENSIONS:
55         return jsonify({"error": "File extension not allowed"}), 400
56
57     head = file.stream.read(8192)
58     if not head:
59         return jsonify({"error": "Empty file"}), 400
60
61     kind = detect_kind(head)
62     if kind in ("exe", "zip", "binary", "unknown"):
63         return jsonify({"error": "Unsafe or unsupported file content"}), 400
64
65     if kind not in EXTENSION_TO_KIND[ext]:
66         return jsonify({"error": "File content does not match extension"}), 400
67
68     file.stream.seek(0)
69     stored_name = f"{uuid.uuid4().hex}_{original_name}"
70     destination = UPLOAD_DIR / stored_name
71     file.save(destination)
72
73     return jsonify({"message": "Upload successful", "file": stored_name}), 201
74
75 @app.errorhandler(413)
76 def file_too_large():
77     return jsonify({"error": "File too large. Max size is 5 MB"}), 413
78
79 if __name__ == "__main__":
80     app.run(debug=False)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\kruth> cd 'c:\Users\kruth'; & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ebugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '57474' '--' 'c:\Users\kruth\AAC A 20.5.py'
ModuleNotFoundError: No module named 'flask'
PS C:\Users\kruth> cd 'c:\Users\kruth'; & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ebugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '57510' '--' 'c:\Users\kruth\AAC A 20.5.py'
* Serving Flask app 'AAC A 20.5'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
PS C:\Users\kruth>
```

Task 4 – Command Injection Prevention

Task:

Analyze an AI-generated Python script that executes system commands.

Instructions:

- Ask AI to generate code using `os.system()` or `subprocess`.
- Identify command injection risks with user input.
- Fix using safe subprocess calls and input validation.

Expected Output:

- A secure command execution script.

```
AAC A 20.5.py 2 X
C:\Users\kruth > AAC A 20.5.py > ...
1 import re
2 import subprocess
3
4 ALLOWED_COMMANDS = [
5     "ping", "hostname", ["-n", "-w"], "time", ["-t", "-M"]],
6     "netstat", ["-b", "-n"], "ipconfig", []],
7 ]
8 SAFE_TARGET_PATTERN = re.compile(r"^[a-zA-Z0-9._]{1,255}$")
9
10 def validate_target(target: str) -> str:
11     value = target.strip()
12     if not SAFE_TARGET_PATTERN.fullmatch(value):
13         raise ValueError("Invalid target")
14     if value.startswith("/") or value.endswith("/") or "." in value:
15         raise ValueError("Invalid target")
16     return value
17
18 def validate_count(count: str) -> int:
19     value = int(count)
20     if value < 1 or value > 10:
21         raise ValueError("Count must be between 1 and 10")
22     return value
23
24 def run_safe_command(target: str, count: int = 4) -> str:
25     cmd = command.strip().lstrip()
26     if not cmd in ALLOWED_COMMANDS:
27         raise ValueError("Unsupported command")
28     safe_target = validate_target(target)
29     safe_count = validate_count(count)
30     if cmd == "ping":
31         args = ["ping", "-n", safe_count, safe_target]
32     else:
33         args = ["hostname", safe_target]
34
35     result = subprocess.run(args, capture_output=True, text=True)
36     return result.stdout.strip()
37
38 if __name__ == "__main__":
39     app.run(debug=False)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\kruth> cd 'c:\Users\kruth'; & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ebugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '57510' '--' 'c:\Users\kruth\AAC A 20.5.py'
Command (ping) started: ping
Ping count (3.00, -n) 4.0
Pinging google.com [142.250.182.78] with 32 bytes of data:
Reply from 142.250.182.78: bytes=32 time=20ms TTL=116
Reply from 142.250.182.78: bytes=32 time=20ms TTL=116
Reply from 142.250.182.78: bytes=32 time=20ms TTL=116
Reply from 142.250.182.78: bytes=32 time=20ms TTL=116
```